

Clean Data

Data Analysis & Information Extraction

Table of contents

Data Wrangling	2
Cleaning Data	2
Factors	4
Shirt sizes data	4
Changing levels	6
Reordering the levels	7
Dates & Time	12
Parsing dates	12
make_datetime()	13
Change formats	14
Extracting components	15
Modifying components	16
Computations with dates and time	17
String	21
Basic string functions	21
Regular Expressions	23

Data Wrangling

Objective: Make data ready for their future **exploration** and **modelling**

We need to convert **raw data** into **processed data**

Raw Data

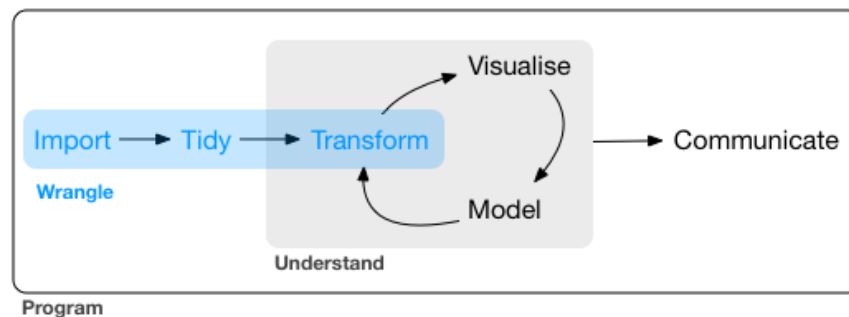
- Data as it appears on the source origin.
- Any manipulation has been done to them.

Processed Data

- Each variable in a column.
- Each observation in a row.
- Each observational unit is a cell.
- More complex data, interconnected tables.

The process work as follows...

1. Importing the data
2. Organizing the data
3. Transforming the data



Cleaning Data

We have learnt how to import and organize data structure for the analysis, but now we have to know how to transform the data to suit our needs and make mutations to clean it more and have it ready for our analytically purpose.

We will follow focus on three groups of data...

Cleaning Data

- Strings
- Factors
- Dates and time

Packages

For this part we are going to work with three data types, for each data type there is an special package implemented in the **tidyverse** ecosystem.

- `stringr`, For working with Strings
- `forcats`, For working with factors (categorical data)
- `lubridate`, For working with dates and times.

Factors

When working with categories, normally texts that represent certain categories of information we will need to them follow certain rules and validations.

With factors we can solved different issues and maintain categorical values:

- **Validation:** Avoid values not existent in the correspondent category.
- **Order:** Make them follow certain order depending on their significance.

As seen in the start of the unit we will work with forcats package included in the tidyverse environment.

```
library(tidyverse) # use library(forcats) for this package only
```

Shirt sizes data

We are going to read the following data set with information of different shirts sizes sales...

```
tshirt_data <- read_csv("https://raw.githubusercontent.com/carviagu/data4courses/refs/heads/main/
```

```
Rows: 766 Columns: 3
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): brand, size
```

```
dbl (1): size_num
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
tshirt_data %>%
  count(size) %>%
  arrange(desc(n))
```

```
# A tibble: 8 x 2
```

```
  size      n
<chr> <int>
1 L      161
2 M      161
3 XL     158
4 S      151
```

Shirt sizes data

5	XXL	105
6	XS	28
7	3XL	1
8	XXS	1

In the following data set we have information of number of shirts sold with the size code. This sizes follow a natural order as the lowest one will be XXS and the highest one will be XXL.

But, if we try order the data by the size code something happens...

```
tshirt_data %>%
  count(size) %>%
  arrange(size)
```

```
# A tibble: 8 x 2
  size      n
<chr> <int>
1 3XL      1
2 L      161
3 M      161
4 S      151
5 XL      158
6 XS       28
7 XXL     105
8 XXS       1
```

The size codes don't follow their natural order as them are being sorted by character following the order of the letters.

We can fix this converting the size code to a factor... like this example...

```
# First we define the size codes order
size_levels <- c("XXS", "XS", "S", "M", "L", "XL", "XXL")

tshirt_data_2 <- tshirt_data %>%
  mutate(
    # We transform the character variable to a factor
    # providing the levels order
    size = factor(size, levels = size_levels)
  )

# Now we can see how the order changes
```

```
tshirt_data_2 %>%
  count(size) %>%
  arrange(size)
```

```
# A tibble: 8 x 2
  size      n
  <fct> <int>
1 XXS      1
2 XS      28
3 S      151
4 M      161
5 L      161
6 XL     158
7 XXL     105
8 <NA>      1
```

But something has changed too, we have NAs. This happens as we have defined the possible levels for the factor, and there are some categories that doesn't fit the levels defined, when doing the transformation, this values are set to NA as they are no valid.

You can check the possible levels we have with `levels` function:

```
levels(tshirt_data_2$size)
```

```
[1] "XXS" "XS"  "S"   "M"   "L"   "XL"  "XXL"
```

Changing levels

You can change the existing levels re-coding them with `fct_recode`. In this example we want that to change XXS and XXL to other names...

```
tshirt_data_2 %>%
  mutate(
    size = fct_recode(size,
                      "Less than XS" = "XXS",
                      "Higher than XXL" = "XXL")
  ) %>%
  count(size)
```

Reordering the levels

```
# A tibble: 8 x 2
  size          n
  <fct>        <int>
1 Less than XS      1
2 XS                28
3 S                151
4 M                161
5 L                161
6 XL               158
7 Higher than XXL  105
8 <NA>              1
```

We can also collapse them, creating a higher hierarchy with `fct_collapse`

```
tshirt_data_2 %>%
  mutate(
    size_group = fct_collapse(size,
      regular = c("XS", "S", "M", "L", "XL"),
      small = c("XXS"),
      large = c("XXL"))
  ) %>%
  count(size_group, size)
```

```
# A tibble: 8 x 3
  size_group size          n
  <fct>        <fct> <int>
1 small      XXS          1
2 regular    XS          28
3 regular    S          151
4 regular    M          161
5 regular    L          161
6 regular    XL          158
7 large      XXL          105
8 <NA>        <NA>          1
```

Reordering the levels

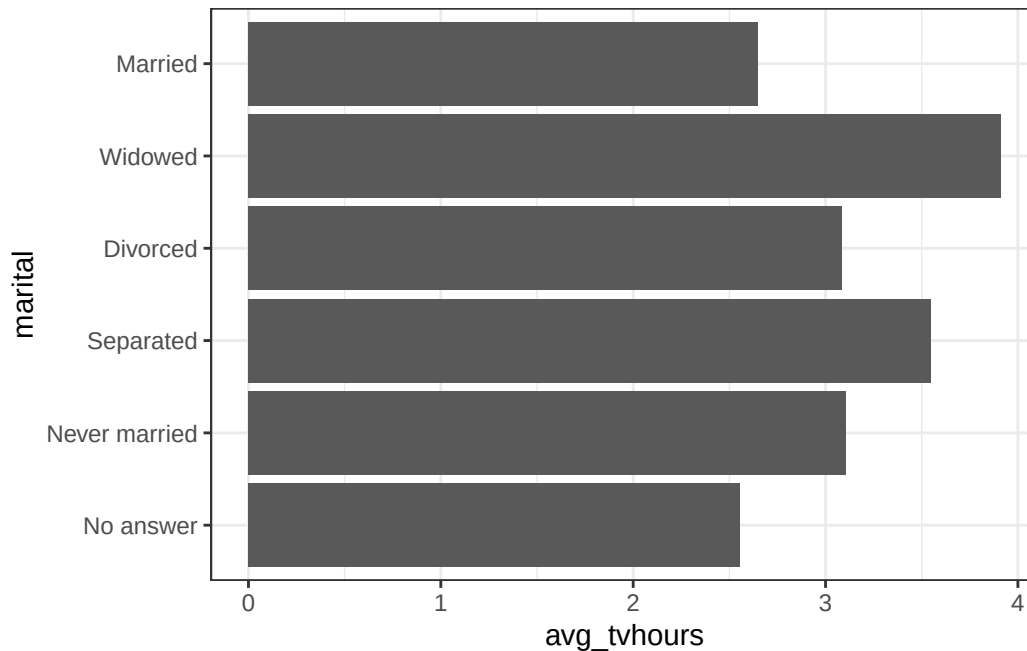
Other useful approach, can be reordering the factors following another value. This is frequently done when showing the the data in visualizations...

```
gss_cat
```

```
# A tibble: 21,483 x 9
  year marital      age race  rincome      partyid  relig denom tvhours
  <int> <fct>      <int> <fct> <fct>      <fct>      <fct> <fct>      <int>
1  2000 Never married    26 White $8000 to 9999 Ind,near ~ Prot~ Sout~    12
2  2000 Divorced        48 White $8000 to 9999 Not str r~ Prot~ Bapt~    NA
3  2000 Widowed        67 White Not applicable Independe~ Prot~ No d~     2
4  2000 Never married    39 White Not applicable Ind,near ~ Orth~ Not ~     4
5  2000 Divorced        25 White Not applicable Not str d~ None Not ~     1
6  2000 Married        25 White $20000 - 24999 Strong de~ Prot~ Sout~    NA
7  2000 Never married    36 White $25000 or more Not str r~ Chri~ Not ~     3
8  2000 Divorced        44 White $7000 to 7999 Ind,near ~ Prot~ Luth~    NA
9  2000 Married        44 White $25000 or more Not str d~ Prot~ Other     0
10 2000 Married        47 White $25000 or more Strong re~ Prot~ Sout~     3
# i 21,473 more rows
```

We can show a plot using counting the average tvhours per marital status...

```
gss_cat %>%
  group_by(marital) %>%
  summarise(avg_tvhours = mean(tvhours, na.rm=T)) %>%
  ggplot() +
  geom_col(aes(x =avg_tvhours, y=marital)) +
  theme_bw()
```

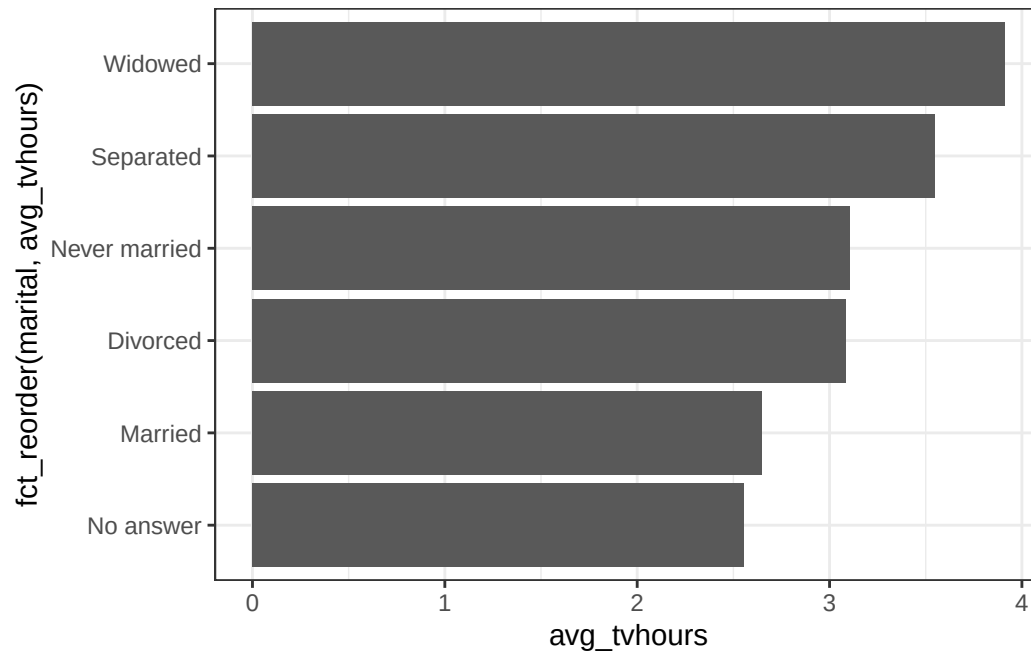



Visually is not so easy to analyze, but we can change the order of the results so the categories are show from higher to lower average hours. For doing it we can use `fct_reorder` function with three arguments:

- **f**, the factor we want to reorder.
- **x**, numeric vector for reordering.
- Optional. **fun**, in the case that there is different **x** values for the same **f**. By default, **fun** is the average.

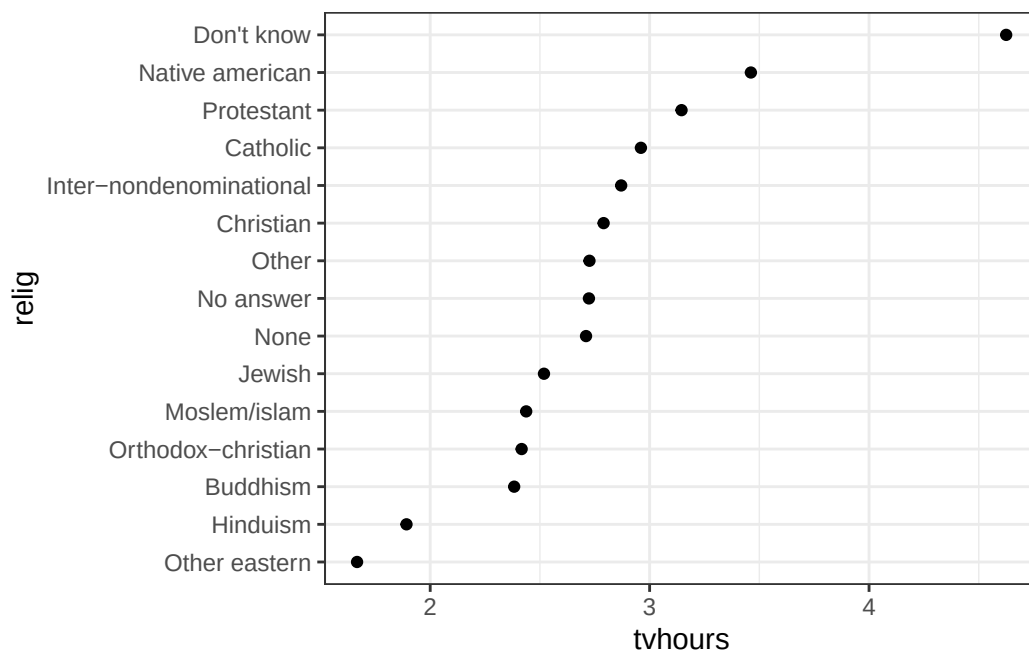
```
gss_cat %>%  
  group_by(marital) %>%  
  summarise(avg_tvhours = mean(tvhours, na.rm=T)) %>%  
  ggplot() +  
  geom_col(aes(x =avg_tvhours, y=fct_reorder(marital, avg_tvhours))) +  
  theme_bw()
```

Reordering the levels



There is a second way of using the `fct_reorder` in a `mutate`...

```
gss_cat %>%
  group_by(relig) %>%
  summarise(tvhours = mean(tvhours, na.rm=TRUE)) %>%
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) +
  geom_point() + theme_bw()
```



Dates & Time

As shown, for working with dates and times we are going to use `lubridate` package. It is now a part of `tidyverse` and is included when you load it, but in older R versions could not be included and you will need to load it individually.

```
library(tidyverse) # use library(lubridate) for this package only
```

We can see three data types...

- Dates
- Time
- Datetime, combination of the first two identifying a time period. Take into account that in tibbles they are identified as `datetime (dtm)` but R base uses `POSIXct` as normal type.

We can get today's date...

```
today()
```

```
[1] "2026-08-30"
```

Or current date time...

```
now()
```

```
[1] "2026-08-30 15:58:57 CEST"
```

Parsing dates

Normally we will receive this data as Strings, this will make us think use the parsing functions we learn at the first block of this unit, but...

`lubridate` has a more easy way of doing it with multiple purpose functions where we just need to specify the order with **y**, **m** or **d**

```
ymd("2022-05-04")
```

```
[1] "2022-05-04"
```

`make_datetime()`

```
ymd("22-May-04")
```

```
[1] "2022-05-04"
```

More examples, the syntax offered by lubridate has a lot of possibilities to parse dates and times:

```
mdy("May 4th, 2022")
```

```
[1] "2022-05-04"
```

```
dmy("4 of May of 2022")
```

```
[1] "2022-05-04"
```

```
ymd_hms("2022-05-04 20:11:58")
```

```
[1] "2022-05-04 20:11:58 UTC"
```

```
mdy_hm("04/05/2022 00:01")
```

```
[1] "2022-04-05 00:01:00 UTC"
```

make_datetime()

Some times we can receive the data in separate variables...

```
library(nycflights13)
flights %>%
  select(year, month, day, hour, minute)
```

```
# A tibble: 336,776 x 5
   year month   day hour minute
  <int> <int> <int> <dbl> <dbl>
1  2013     1     1     5     15
2  2013     1     1     5     29
3  2013     1     1     5     40
4  2013     1     1     5     45
```

Change formats

```
5 2013      1      1      6      0
6 2013      1      1      5     58
7 2013      1      1      6      0
8 2013      1      1      6      0
9 2013      1      1      6      0
10 2013     1      1      6      0
# i 336,766 more rows
```

We can simply create date with individual variables using `make_datetime()`.

```
flights %>%
  select(year, month, day, hour, minute) %>%
  mutate(departure = make_datetime(year, month, day, hour, minute)) %>%
  head(5)
```

```
# A tibble: 5 x 6
   year month   day hour minute departure
<int> <int> <int> <dbl> <dbl> <dtm>
1  2013     1     1     5     15 2013-01-01 05:15:00
2  2013     1     1     5     29 2013-01-01 05:29:00
3  2013     1     1     5     40 2013-01-01 05:40:00
4  2013     1     1     5     45 2013-01-01 05:45:00
5  2013     1     1     6      0 2013-01-01 06:00:00
```

Change formats

You can change between formats...

```
as_datetime(today())
```

```
[1] "2026-08-30 UTC"
```

```
as_date(now())
```

```
[1] "2026-08-30"
```

Extracting components

You can extract components for dates and time...

```
datetime <- ymd_hms("2022-05-04 12:34:56")  
year(datetime)
```

```
[1] 2022
```

```
month(datetime)
```

```
[1] 5
```

```
mday(datetime) # day of month
```

```
[1] 4
```

```
yday(datetime) # day of year
```

```
[1] 124
```

```
wday(datetime) # day of week
```

```
[1] 4
```

```
hour(datetime)
```

```
[1] 12
```

```
minute(datetime)
```

```
[1] 34
```

```
second(datetime)
```

```
[1] 56
```

In these examples the extractions are numerical. So we have the day, hour etc. But for some date attributes (like day of week or month) we have a name. We change the behavior of the functions to obtain the label.

Modifying components

```
datetime <- ymd_hms("2022-05-04 12:34:56")
month(datetime, label = TRUE)
```

```
[1] May
12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

```
wday(datetime, label = TRUE, abbr = FALSE)
```

```
[1] Wednesday
7 Levels: Sunday < Monday < Tuesday < Wednesday < Thursday < ... < Saturday
```

You can change labels output language.

```
wday(datetime, label = TRUE, abbr = FALSE, locale = Sys.setlocale("LC_TIME", "es_ES"))
```

```
[1] miércoles
7 Levels: domingo < lunes < martes < miércoles < jueves < ... < sábado
```

Modifying components

We can modify the components using the same functions

```
datetime <- ymd_hms("2022-05-04 12:34:56")
year(datetime) <- 2023
datetime
```

```
[1] "2023-05-04 12:34:56 UTC"
```

```
month(datetime) <- 01
datetime
```

```
[1] "2023-01-04 12:34:56 UTC"
```

```
hour(datetime) <- hour(datetime) + 1
datetime
```

```
[1] "2023-01-04 13:34:56 UTC"
```


Computations with dates and time

For computing with dates we need...

- **Duration**, exact number of seconds
- **Periods**, other units like months and weeks.

Duration

When making the difference between two dates, will obtain a `difftime` object...

```
my_age <- today() - ymd("1998 Mar 4")  
my_age
```

Time difference of 10406 days

```
class(my_age)
```

```
[1] "difftime"
```

This object will contain the information regarding time period in seconds, minutes, hours, days, etc... pretty ambiguous.

Alternative, we can work with **duration** (in seconds)

```
as.duration(my_age)
```

```
[1] "899078400s (~28.49 years)"
```

Other useful functions... in seconds

```
dseconds(15)
```

```
[1] "15s"
```

```
dminutes(10)
```

```
[1] "600s (~10 minutes)"
```

```
dhours(c(12, 24))
```

```
[1] "43200s (~12 hours)" "86400s (~1 days)"
```

```
ddays(0:5)
```

```
[1] "0s" "86400s (~1 days)" "172800s (~2 days)"  
[4] "259200s (~3 days)" "345600s (~4 days)" "432000s (~5 days)"
```

```
dweeks(3)
```

```
[1] "1814400s (~3 weeks)"
```

```
dyears(1)
```

```
[1] "31557600s (~1 years)"
```

Making computations in seconds...

```
dyears(1) + dweeks(12) + dhours(15)
```

```
[1] "38869200s (~1.23 years)"
```

```
2 + dweeks(12)
```

```
[1] "7257602s (~12 weeks)"
```

```
tomorrow <- today() + ddays(1)  
last_year <- today() - dyears(1)
```

Periods

Last methods will help making computations using seconds as our base... but we have other functions more intuitive for just operating with the exact period...

```
seconds(15)
```

```
[1] "15S"
```

```
minutes(10)
```

```
[1] "10M 0S"
```

```
hours(c(12, 24))
```

```
[1] "12H 0M 0S" "24H 0M 0S"
```

```
days(7)
```

```
[1] "7d 0H 0M 0S"
```

```
months(1:6)
```

```
[1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"  
[5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
```

```
weeks(3)
```

```
[1] "21d 0H 0M 0S"
```

```
years(1)
```

```
[1] "1y 0m 0d 0H 0M 0S"
```

Computing...

```
ymd("2022-01-01") + years(1)
```

```
[1] "2023-01-01"
```

```
ymd("2022-01-01") + months(1)
```

```
[1] "2022-02-01"
```

```
ymd("2022-01-01") + days(1)
```

```
[1] "2022-01-02"
```

Take care when mixing both systems... or periods or duration not both...

```
ymd("2022-01-01") + dyears(1)
```

```
[1] "2023-01-01 06:00:00 UTC"
```

```
ymd("2022-01-01") + years(1)
```

```
[1] "2023-01-01"
```

String

Remember an String is a text format here some examples...

```
"This is a String"
```

```
c("one", "two and three")
```

For doing all that we will use `stringr` a tidyverse package developed to work with Strings. All the functions in this package start with `str_`

```
library(tidyverse) # use library(stringr) for this package only
```

Basic string functions

Length of a string

We can compute the length of an string using `str_length()`

```
str_length("Hello")
```

```
[1] 5
```

All operations can be applied to vectors too...

```
str_length(c("Hello", "World!"))
```

```
[1] 5 6
```

What will be the behavior with blank spaces? Blank spaces are consider characters too, so they will count on the total.

```
str_length("CUNEF EDU")
```

```
[1] 9
```

Subsetting strings

You can extract parts of strings...

```
str_sub("Hello World",start = 1, end = 5)
```

```
[1] "Hello"
```

Or by the end...

```
str_sub("Hello World",start = -5, end = -1)
```

```
[1] "World"
```

Changing case

You can change the case of the letters. There are several options...

- All to lower case with `str_to_lower`
- All to upper case with `str_to_upper`
- All to title case with `str_to_title`, first letter of each word in upper case, rest on lower case.

```
str_to_lower("HeLlo wOrld")
```

```
[1] "hello world"
```

```
str_to_upper("HeLlo wOrld")
```

```
[1] "HELLO WORLD"
```

```
str_to_title("HeLlo wOrld")
```

```
[1] "Hello World"
```

Concatenating Strings

We can combine (concatenate) Strings using `str_c()`

```
str_c("Va", "len", "cia")
```

```
[1] "Valencia"
```

You can specify the separation string used when doing the concatenation with the `sep` argument.

```
str_c("One", "two", sep=",")
```

```
[1] "One,two"
```

This works with vectors too...

```
str_c("prefix-", c("X", "Y", "Z"), "-sufix")
```

```
[1] "prefix-X-sufix" "prefix-Y-sufix" "prefix-Z-sufix"
```

For collapsing a full large text in a single string with collapse argument.

```
str_c(c("A", "long", "time", "ago,",
        "in", "a", "galaxy", "far", "far", "away..."),
      collapse = " ")
```

```
[1] "A long time ago, in a galaxy far far away..."
```

Regular Expressions

Regular expressions (*regexp*) are tools designed to find patterns in texts. We can do several things with patterns...

- Find which strings complain with the pattern.
- Find in which position appears the pattern.
- Extract the pattern from the string.
- Replace the pattern in the string.

For example, we want to detect ES followed by two numbers”

```
x1 <- "ES49 "
x2 <- "Mi account number is ES34 30..."
x3 <- "I live in Spain (ES)"

str_view_all(c(x1,x2,x3), 'ES[0-9]{2}')
```

```
[1] | <ES49>
[2] | Mi account number is <ES34> 30...
[3] | I live in Spain (ES)
```

Regular Expressions

In the example we have created a pattern 'ES[0-9]{2}' to match the structure we need to find 'ES' followed by two numbers. We do a search on the examples and we can see that the pattern has been found in the two first examples.

i In the example seen before, we have use `str_view`. This function is used to validate patterns and for learning purposes only.

For building patterns, R has a wide range of tools we can use to develop the proper pattern syntax. General rules for building them are...

- exact word or text we want to find.
- . identifies any character
- \\s blank space
- [] set of elements e.g. [abc]: a or b or c
- [0-9] all numbers from 0 to 9
- [a-z] all letters in lower case
- [A-Z] all letters in upper case
- We can use + to repeat a pattern several times, e.g. two numbers: [0-9][0-9] or [0-9]+ will work.
- ab|bc two possible valid options ab or bc
- ^ start pattern at the beginning
- \$ start pattern at the end

You test and try *regex* in the following [link](#)

We will review all of them with function examples...

Finding patterns: `str_detect`

For trying to find if a text complies with a pattern we can use `str_detect` function.

Imagine that we want to find the words that contain Le (**exact match**) in the following list.

```
x <- c("Cats", "Leia Organa", "Capital", "Leman")
str_detect(x, "Le")
```

```
[1] FALSE TRUE FALSE TRUE
```

The output of `str_detect` is a logical value. TRUE if the pattern appears and FALSE otherwise.

So, how to use it properly on data analysis. Imagine the following example...


```
sample_tb <- tibble(  
  "texts" = c("234H",  
              "B12, C15, E14",  
              "Academic Year 1",  
              "Two and a half Men",  
              "Account ES14 2345 5678 2456")  
)  
  
sample_tb
```

```
# A tibble: 5 x 1  
  texts  
  <chr>  
1 234H  
2 B12, C15, E14  
3 Academic Year 1  
4 Two and a half Men  
5 Account ES14 2345 5678 2456
```

We want to find all texts that contain a letter a. We will do it using filter...

```
sample_tb %>%  
  filter(str_detect(texts, "a"))
```

```
# A tibble: 2 x 1  
  texts  
  <chr>  
1 Academic Year 1  
2 Two and a half Men
```

The detect is only looking for exact match of letter a, let's try to find any one that starts with letter a

For that we need to use \$ symbol

```
sample_tb %>%  
  filter(str_detect(texts, "^a"))
```

```
# A tibble: 0 x 1  
# i 1 variable: texts <chr>
```

Looks nothing is returned, maybe is due to the case, we can solve it two ways...

```
sample_tb %>%  
  filter(str_detect(texts, "^a|^A"))
```

```
# A tibble: 2 x 1  
  texts  
  <chr>  
1 Academic Year 1  
2 Account ES14 2345 5678 2456
```

```
sample_tb %>%  
  filter(  
    str_detect(  
      str_to_lower(texts), "^a")
```

```
# A tibble: 2 x 1  
  texts  
  <chr>  
1 Academic Year 1  
2 Account ES14 2345 5678 2456
```

Last example, we want any text with a combination of two or more numbers...

```
sample_tb %>%  
  filter(str_detect(texts, "[0-9][0-9]+"))
```

```
# A tibble: 3 x 1  
  texts  
  <chr>  
1 234H  
2 B12, C15, E14  
3 Account ES14 2345 5678 2456
```

Counting number of appearances: `str_count`

We can count the times the pattern is detected with `str_count`

For example, count the number of a in each text.

```
sample_tb %>%
  mutate(
    as = str_count(texts, "a")
  )
```

```
# A tibble: 5 x 2
  texts          as
  <chr>        <int>
1 234H            0
2 B12, C15, E14    0
3 Academic Year 1    2
4 Two and a half Men 3
5 Account ES14 2345 5678 2456 0
```

Replacing: str_replace

Another usage of patterns is to detect them and make replacements, for example for treating data and replace it. We can do this with `str_replace()` or `str_replace_all()`.

In the example we want to replace the blank spaces by _

```
sample_tb %>%
  mutate(
    text_clean = str_replace(texts, "\\s", "_")
  )
```

```
# A tibble: 5 x 2
  texts          text_clean
  <chr>        <chr>
1 234H        234H
2 B12, C15, E14 B12,_C15, E14
3 Academic Year 1 Academic_Year 1
4 Two and a half Men Two_and a half Men
5 Account ES14 2345 5678 2456 Account_ES14 2345 5678 2456
```

By default it only replaces one match, if we want to apply it to every coincidence we need to use `str_replace_all`

```
sample_tb %>%
  mutate(
    text_clean = str_replace_all(texts, "\\s", "_")
  )
```

Regular Expressions

```
# A tibble: 5 x 2
  texts                text_clean
  <chr>                <chr>
1 234H                234H
2 B12, C15, E14      B12,_C15,_E14
3 Academic Year 1    Academic_Year_1
4 Two and a half Men Two_and_a_half_Men
5 Account ES14 2345 5678 2456 Account_ES14_2345_5678_2456
```

Extracting: `str_extract`

We can also extract patterns matched...

In the example we extract any combination of a letter+number:

```
sample_tb %>%
  mutate(
    letter_num = str_extract(texts, "[A-Z][0-9]")
  )
```

```
# A tibble: 5 x 2
  texts                letter_num
  <chr>                <chr>
1 234H                <NA>
2 B12, C15, E14      B1
3 Academic Year 1    <NA>
4 Two and a half Men <NA>
5 Account ES14 2345 5678 2456 S1
```